

n-step TD bootstrapping

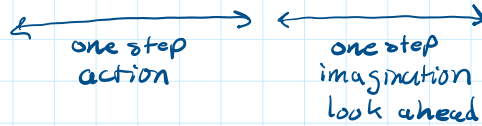
MC



Temporal difference

We look at the entire episode,  $t \leq T$

TD



TD(0) algo

$V_\pi$  estimation

$$V(s) \leftarrow V(s) + \alpha [r + \sqrt{V(s')} - V(s)] \quad \equiv \quad V(s) \leftarrow V(s) + \alpha [G_{t:t+1} - V(s)]$$

↑  
action gives reward
↑  
look at next state

TD error
TD error

SARSA

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \sqrt{Q(s',a')} - Q(s,a)] \quad \equiv \quad Q(s,a) \leftarrow Q(s,a) + \alpha [G_{t:t+1} - Q(s,a)]$$

TD error
TD error

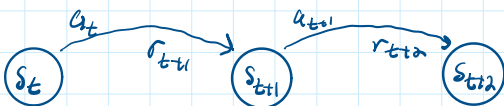
Q-learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \sqrt{\max_{a'} Q(s',a')} - Q(s,a)]$$

TD error

Why only look ahead more than one step

TD(1) algo or 2-step actions/look ahead



$$G_{t:t+2} \Big|_s = r_{t+1} + \sqrt{r_{t+2}} + \sqrt{V(s_{t+2})}$$

$$G_{t:t+2} \Big|_{s,a} = r_{t+1} + \sqrt{r_{t+2}} + \sqrt{Q(s_{t+2}, a_{t+2})}$$

looking at algo on figure 2.1

also n-step TD for estimating  $V \approx V_\pi$

also n-step Sarsa for estimating  $Q \approx q_*$  or  $q_\pi$

also off policy n-step Sarsa for estimating  $Q \approx q_*$  or  $q_\pi$

Lecture on this chapter is done - need to still go over the rest of the chapter on our own

## Machine learning start lecture

### Traditional RL

- states are assumed simple indices

$s_1, s_2, \dots, s_n$

we may want better representations of states

Professor drew a parking lot and described an agent that had limited view of the parking areas, only has line of sight

Parking lot is a grid structure

What can the agent see based on the current state it is in, full parking spaces, empty ones, etc....

- There is no memory or history of what happened in the past

✓ Quite likely an agent remembers the recent past

✓ Agent has detailed past memory of past  $n$  episodes

- Actions are discrete, up, down, left, right

actions in real life are often parameterizable

- When state representations become complex

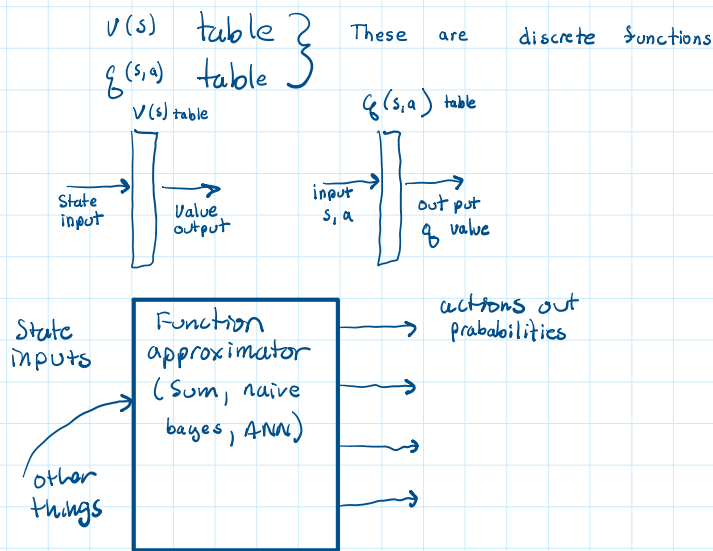
we need more complex machinery or computing algos to augment the RL process

professor drew Tetris representation

- The number of states becomes large

- RL agent may have to perform well in states it has never seen

- RL agent may have to perform well in states it has never seen



Machine learning finding the functions based on a table of inputs and outputs

| $x_1$ | $x_2$ | Function | it should be able to figure this function adds the inputs $x_1$ and $x_2$<br>easy for linear almost impossible for non-linear | it is also possible that there are errors in the training data that the ML should still be able to figure out. |
|-------|-------|----------|-------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| 1     | 2     | 3        |                                                                                                                               |                                                                                                                |
| 2     | 2     | 4        |                                                                                                                               |                                                                                                                |
| 10    | 1     | 11       |                                                                                                                               |                                                                                                                |